



Examen Final PCLP1

Student: _____ Grupa: _____

Descriere curs:	PCLP1, An I, Semestrul I	Rezultate Examen	
Titlu curs:	Programarea Calculatoarelor și Limbaje de Programare 1	Subiect	Punctaj
Profesor:	Prof.dr.ing. Florin POP	1	/5
Data examen:		2	/5
Durata examen:	2 ore	3	/1
Tip Examen:	"Closed Book"	Σ	/11
Numar pagini:	_____		

5 puncte

1. Data filtering. Gigel, desigur, dorește să filtreze liniile unui text în funcție de anumite cuvinte cheie. El scrie un program în **C** care primește ca parametri n chei (maxim 24), cuvinte de lungime `MAX_WORD_SIZE`, de forma:

```
./data_filter key_1 key_2 ... key_n
```

Apoi, citește de la mediul standard de intrare liniile textului, fiecare având lungimea maxim posibilă de `MAX_LINE_SIZE` caractere. Textul se termină cu o linie care conține doar caracterul punct ('.'). Filtrarea se face "on the fly", adică în timpul citirii, astfel:

- dacă linia nu conține nicio cheie, aceasta se scrie așa cum este într-un fișier text cu numele `data.txt`, care se creează în timpul execuției, doar dacă este nevoie;
- dacă linia conține cel puțin o cheie, aceasta (linia) se concatenează la un șir (denumit *text agregat*) care este re-alocat dinamic la fiecare concatenare și se contorizează frecvența de apariție a fiecărei chei; când *textul agregat* ar putea depăși dimensiunea maximă de `MAX_TEXT_SIZE` acesta se scrie într-un fișier text, cu numele `filtered_data.txt` (creat la prima operație de scriere necesară) împreună cu cheile și frecvențele de apariție ale acestora, resetându-se toate datele interne folosite (*textul agregat*, contoarele, etc.).

La finalizarea citirii textului de intrare, toate datele rămase în memorie se scriu în fișier (*textul agregat*, cheile și frecvențele lor), se eliberează memoria și se închid fișierele.

Restricții și precizări: Se cere să se gestioneze eficient memoria **DOAR** pentru textul agregat.

Punctare: Soluția corectă implementată valorează 3p. Se va acorda: 1p pentru modularizare; 1p pentru aspecte legate de gestionarea lucrului eficient cu memoria și fișierele.

5 puncte

2. **Data recovery.** Gigel scrie un program **C** pentru recuperarea unor fișiere șterse definitiv de pe un SSD. El a aflat că, dacă nu s-a întâmplat să suprascrie sectoarele de pe disc cu alte fișiere, ar putea să le recupereze. Pentru simplitate, considerăm că un SSD este un **fișier binar** numit "**ssd.bin**", iar pozele corespund unor structuri scrise în acest fișier.

Vom considera un singur tip de fișiere de pe SSD care trebuie recuperat: GPEG (Gigel Photographic Experts Group). O poză în format GPEG este un fișier care are un **nume** (de lungime variabilă, număr natural pe 1 byte), o **calitate** (număr real din $[0, 1]$ pe 64 de biți), dimensiunile **lungime** și **lățime** (numere naturale pe 16 biți), un șir de **4 x lungime x lățime** bytes (șir de pixeli RGBA ai pozei) și 4 bytes reprezentând sumele de control ale canalelor RGBA (câte o sumă de control pentru fiecare canal, prima pentru canalul R, a doua pentru canalul G, etc). Un exemplu este următorul:

```
$ hexdump -C ssd.bin
00000000  00 c8 03 0a 67 69 67 65  6c 2e 67 70 65 67 cd cc  |....gigel.gpeg..|
00000010  cc cc cc cc ec 3f 03 00  02 00 00 00 00 ff ff ff  |.....?.....|
00000020  ff ff ff 00 00 00 ff ff  ff ff ff 00 00 00 ff ff  |.....|
00000030  ff 01 ff ff ff fe 00 c8  03 36 15 00 09 05       |.....6....|
```

Exemplul dat are 62 de bytes în total. Dacă am citi o poză de la cursor 0 cu byte 00 lungimea numelui nu este validă. Următoare încercare, citește c8, adică 200, care este prea mare ca nume de fișier (depășește dimensiunea SSD). Următoarea încercare, găsește un nume valid de lungime 3 (0a 67 69), o calitate validă cu 8 bytes, dar lungimea și lățimea (cd cc și cc cc) sunt din nou prea mari. Se citește apoi numărul 0a (10) și stringul "gigel.gpeg", o nouă calitate pe 8 bytes, dimensiunile 3 (03 00) și 2 (02 00), apoi $4 * 3 * 2 = 24$ bytes: {00, 00, 00, ff, ff, ff, ff, ff, ff, 00, 00, 00, ff, ff, ff, ff, ff, 00, 00, 00, ff, ff, ff, 01} și sumele de control {ff, ff, ff, fe}. Următorii 8 octeți nu mai pot forma o imagine.

Cerința a) Să se implementeze funcția `int is_valid_gpeg(const gpeg_t* gpeg)`, care întoarce 1 dacă poza este validă și 0 altfel. Pentru a valida poza, vom calcula 4 sume de control, câte una pentru fiecare canal RGBA (canalul R are bytes de pe pozițiile 0, 4, 8, 12, ...; canalul G are bytes de pe pozițiile 1, 5, 9, 13, ...; și tot așa) folosind operația XOR pe biți. Poza este validă dacă cele 4 sume de control corespund (sunt egale) cu cei 4 bytes din structură (de la finalul acesteia).

Cerința b) Să se recupereze toate pozele GPEG posibile de pe SSD (deoarece Gigel nu știe unde începe o poză, va încerca toate posibilitățile - de la cursor 0, 1, 2, etc). O poză este recuperată dacă este citită complet și este validă. Aceasta este scrisă în format binar într-un nou fișier, cu numele corespunzător citit în structură (în exemplul dat, "gigel.gpeg"), astfel: se scrie inițial șirul "GPEG" (fără lungime sau terminator de șir), apoi celelalte date care urmează după nume din structura GPEG. În caz de succes, se continuă recuperarea din poziția curentă, altfel Gigel se întoarce la poziția inițială și înaintează o poziție.

Restricții și precizări: Se cere să se gestioneze eficient memoria.

Punctare: Cerința a) - 1p, iar Cerința b) - 2p. Se va acorda: 1p pentru modularizare; 1p pentru aspecte legate de gestionarea lucrului eficient cu memoria și fișierele.

1 puncte

3. [Bonus] Să se scrie funcția `parse()`, în **C** și în **Python** (0.5p + 0.5p), care să primească calea către un fișier text și un număr **k**. Funcția afișează primele sau ultimele **k** linii din fișier (în funcție de semnul lui **k**). **Restricții C:** nu se permite citirea întregului fișier în memorie; o linie are maximum **NMAX** de caractere. **Restricții Python:** nu există.